

Поле IOAI

- **Обмеження часу:** 5 хвилин
- **Сховище:** 5 GB
- **Розмір розв'язку:** `solution.ipynb`, `custom_model.py` ≤ 1 MB разом
- **Попередньо навчені моделі:** немає — навчання з нуля, без інтернету під час оцінювання
- **Базовий бал:** 31.2187

Завдання

Мер Астани хоче прикрасити місто стилізованими логотипами IOAI. Як статистик, він розглядає все — зокрема й логотип — як просторову функцію $F(x, y, \overline{W})$, де $x, y \in [0, 1]$ представляють координати на 2D-площині, а $\overline{W}$ є набором прихованих параметрів, що визначають такі стилістичні атрибути, як кольори та кути нахилу літер.

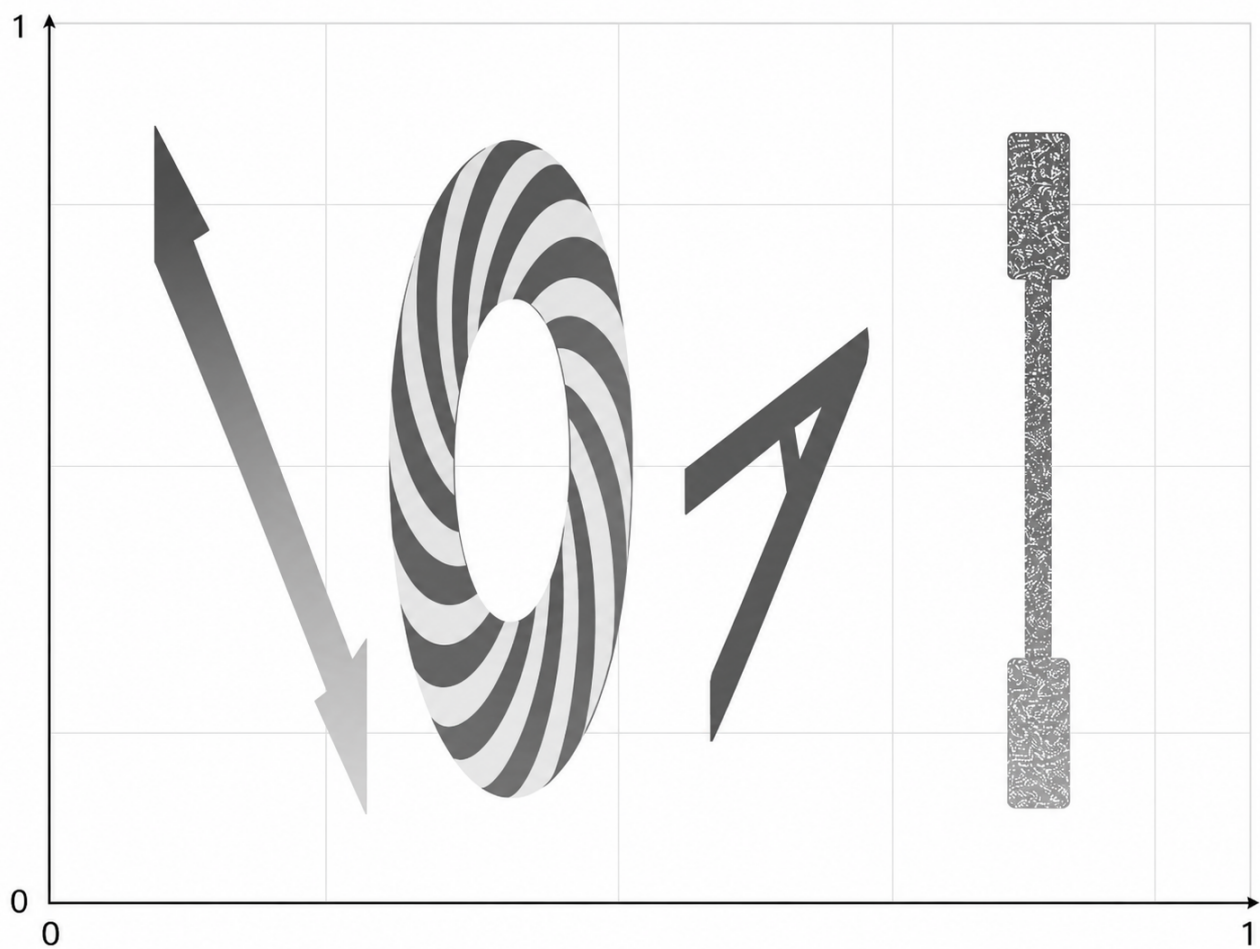
Оскільки F надто складна, щоб виразити її явним математичним рівнянням, ваше завдання — навчити нейронну мережу наближати її. Мережа повертатиме значення **поля IOAI** для будь-якої пари координат (x, y) , створюючи повну візуалізацію логотипа у вигляді теплової карти на всій площині. Нижче наведено приклад візуалізації теплової карти F з деякими конкретними прихованими параметрами $\overline{W}$.

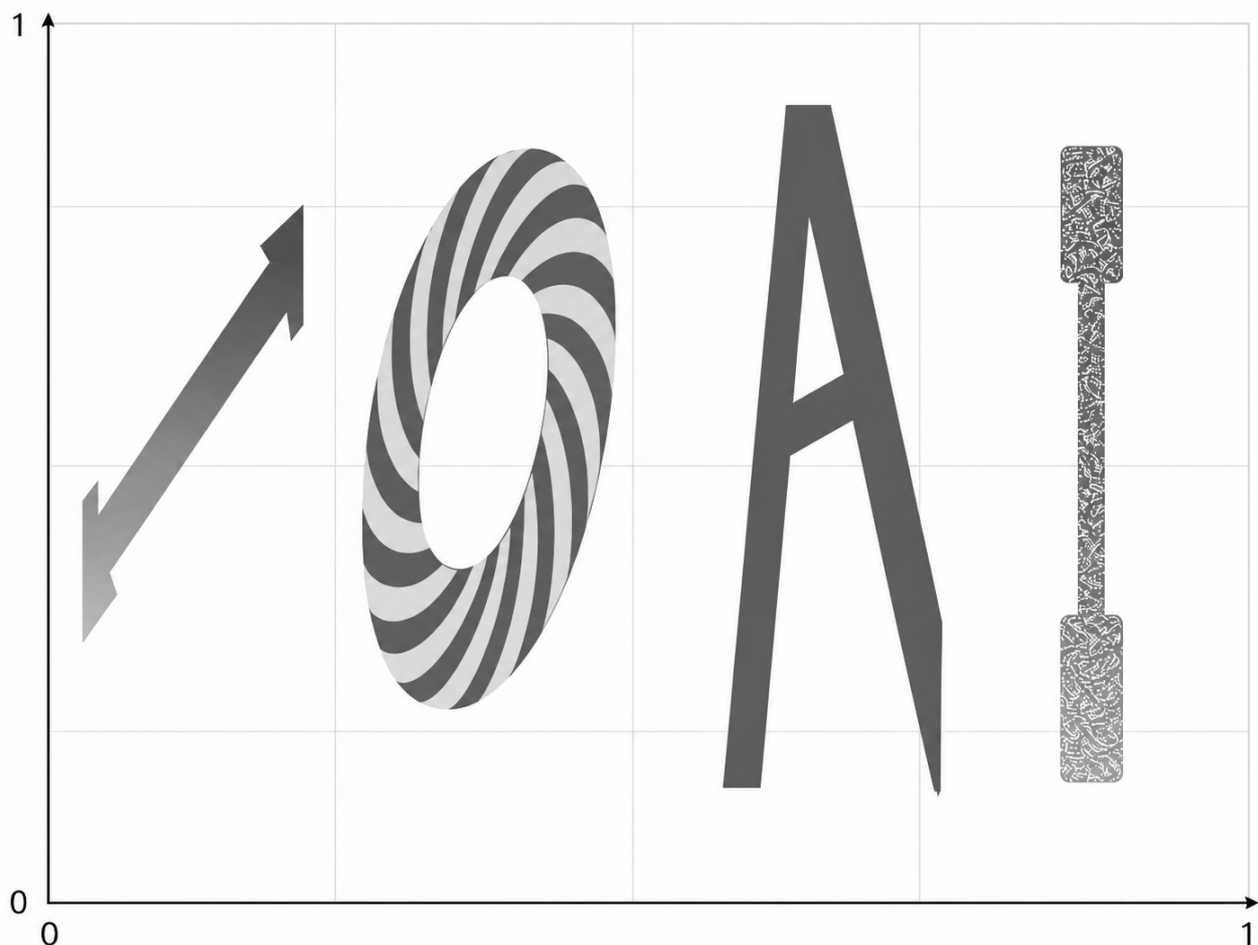


З чого складається поле IOAI? Із чотирьох літер і фону.

- Значення всередині першої літери I дуже великі ($1e+10$ і більше) та мають лінійний градієнт
- Значення в літері O утворюють спіральний візерунок
- Значення всередині літери A завжди дорівнює -1
- Значення всередині останньої літери I мають бути випадковими значеннями з діапазону $[-2026, 2026]$, навіть якщо обчислювати їх у тій самій точці двічі
- Поза літерами значення завжди дорівнює нулю

Функція має приховані параметри $\overline{W}$, які впливають на масштаб і нахил літер, а також на діапазон значень усередині першої літери I . Однак літери не перетинатимуться. Нижче наведено кілька ілюстративних прикладів того, як виглядає поле IOAI з різними $\overline{W}$:





Що вам надано:

Ця задача НЕ містить наборів даних. Натомість вам надано функцію-генератор, налаштовану за допомогою конфігураційного JSON-файлу за шляхом `data/train_config/field_config.json`.

Тестова конфігурація прихована, але має подібний характер. Ваше завдання — виконати апроксимацію на наданому генераторі, використовуючи стільки даних, скільки забажаєте. Ваші «тренувальний» і «тестовий» розподіли генеруються тим самим генератором — ви лише не знаєте, у яких точках (x_i, y_i) вас оцінюватимуть.

Ваше надсилання має складатися з:

- класу моделі для навчання, збереженого як `custom_model.py`. Ця модель має успадковувати клас `torch.nn.Module` і використовувати лише імпорти `torch`. Вона має містити клас `CustomModel`, який використовується в ноутбучі `solution.ipynb`.
- ноутбука `solution.ipynb`, який створить ваги `model.pt`

Оцінювання

Для кожної області мінімальний бал дорівнює 0, а максимальний — 1. Підсумковий бал усереднюється за всіма п'ятьма областями (чотирма — по одній для кожної літери — і фоном) та множиться на 100. Передбачено **штраф за кількість параметрів**:

Якщо ваша модель має більше ніж 20260 параметрів, бал зменшується вдвічі.

Кількість параметрів вимірюється за допомогою `sum(p.numel() for p in model.parameters())`. Ми очікуємо, що ваша модель також працюватиме в стохастичному режимі, причому `PyTorch nn.Dropout` буде частиною моделі.

Для стандартних областей

Для кожної області R (перша літера `I`, `O`, `A`, `Background`) ми оцінюємо модель на $N_R = 512$ тестових точках (x_i, y_i) зі справжніми значеннями v_i і прогнозами $\hat{v}_i$. Як основну метрику ми використовуємо нормалізовану середню абсолютну похибку (MAE). MAE визначається так:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

А нормалізація виконується так:

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

де $s_R > 0$ — константа масштабу.

Для області останньої літери `I`

У цій області **dropout увімкнено під час оцінювання**. Для кожної тестової точки j :

1. Ми запускаємо модель $K = 10$ разів, щоб отримати $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Якщо будь-який результат виходить за межі діапазону $[-2026, 2026]$, тоді $\text{pointScore}(j) = 0$.
3. Інакше обчислюємо стандартне відхилення σ_j для K результатів і перетворюємо його на бал:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

де $s_E > 0$ — фіксована константа масштабу.

Бал за область є середнім значенням за всіма точками області:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

де $N_E = K * N_R$.

Простіше кажучи, що більше різноманіття ви маєте, то вищим буде ваш бал за цю область. **Ви не можете використовувати випадковість у чистому вигляді, зокрема функції PyTorch `rand*` і `_uniform`; випадковість має походити від інференсу з увімкненим dropout.**

Як надіслати

1. Відкрийте `solution.ipynb` і виконайте всі комірки.
2. Покращте модель `CustomModel` у `custom_model.py`
3. Переконайтеся, що ваша остання комірка зберігає модель у файл `model.pt`.
4. На вкладці Git у JupyterLab додайте до індексу, прокоментуйте та закомітьте `solution.ipynb` і `custom_model.py`, а потім виконайте `push`.
5. Поверніться на сторінку Contest і натисніть **Submit**. Коментар до надсилання має збігатися з коментарем із попереднього кроку.